

Лабораторна робота 7

Тема: Встановлення HTTP-статус кодів та функції керування буферизацією виводу.

Зміст роботи

Завдання 1. Динамічне кешування сторінки з урахуванням статус-коду:

Створіть PHP-скрипт **cache_page.php**, який реалізує кешування сторінки залежно від статус-коду відповіді.

Умови:

1. Якщо сторінка віддає статус **200 OK**, її вміст зберігається у файлі кешу **cache.html**.
2. Якщо статус **404 Not Found**, кешування не відбувається, а файл **cache.html** (якщо він існує) видаляється.
3. Якщо користувач відвідує сторінку повторно, і є кеш-файл, вміст віддається з нього без повторної генерації сторінки.

Підказка:

- Використовуйте **ob_start()** для збору вмісту сторінки.
- Використовуйте **ob_get_contents()** для отримання вмісту кешу.
- Використовуйте **file_put_contents()** для запису кешу.
- Використовуйте **file_get_contents()** для видачі кешу.

Лістинг cache_page.php:

```
<?php
$cache_file = "cache.html";

$is_404 = isset($_GET['error']) && $_GET['error'] == '1';

if (!$is_404 && file_exists($cache_file)) {
    echo file_get_contents($cache_file);
    echo "<hr><p style='color: green;'><em>(Ця сторінка завантажена з кешу)</em></p>";
    exit();
}

ob_start();

if ($is_404) {
    http_response_code(404);
    echo "<h1>404 Not Found</h1>";
    echo "<p>На жаль, запитану сторінку не знайдено.</p>";
} else {
```

					ДУ «Житомирська політехніка».26.121.04.07 – Лр7			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Пригорницький О. В.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Ковтун В. В.						1
Керівник								17
Н. контр.							ФІКТ Гр. ВТ-24-1[2]	
Зав. каф.								

```
http_response_code(200);
echo "<h1>Успішна генерація сторінки</h1>";
echo "<p>Це свіжий контент, згенерований сервером.</p>";
echo "<p>Поточний час генерації: " . date('Y-m-d H:i:s') . "</p>";
}

$status_code = http_response_code();

if ($status_code == 200) {
    $content = ob_get_contents();
    file_put_contents($cache_file, $content);
}

elseif ($status_code == 404) {
    if (file_exists($cache_file)) {
        unlink($cache_file);
    }
}

ob_end_flush();
?>
```

Результат:

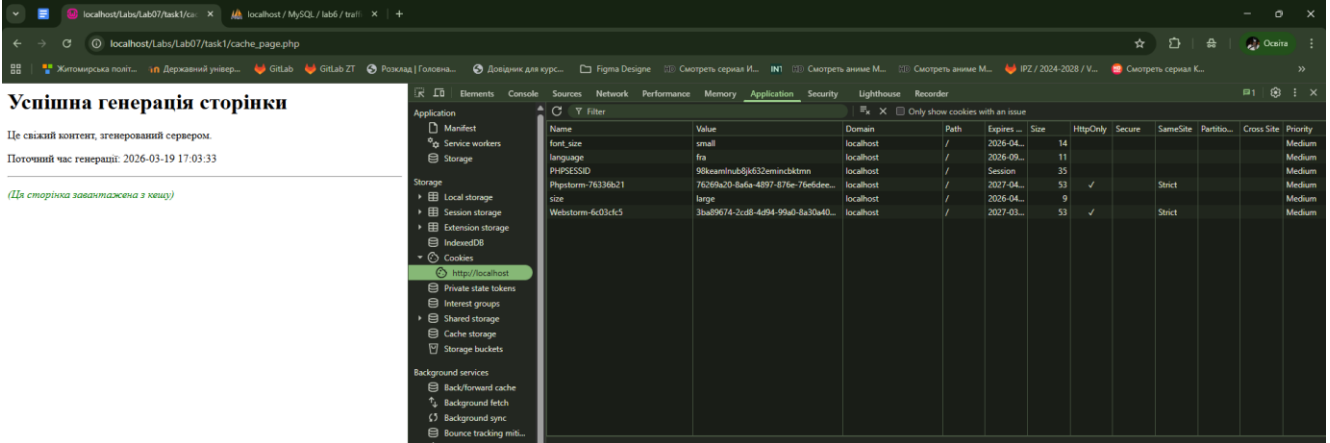


Рис. 1. Завдання 1

Завдання 2. Система захисту від перевантаження сервера (Rate Limiting):

Створіть скрипт **rate_limit.php**, який обмежує кількість запитів від одного користувача та повертає відповідні статус-коди.

Умови:

- Логувати IP-адресу користувача в файл **requests.log**.
- Якщо користувач зробив більше **5 запитів за хвилину**, встановлювати статус **429 Too Many Requests** і рекомендувати спробувати пізніше.
- Якщо ліміт не перевищено, видавати **200 OK** та звичайний контент.

Підказка:

- Використовуйте **ob_start()** для збору відповідей.
- Записуйте в лог: IP, час запиту.
- Видаляйте застарілі записи, щоб не накопичувати зайві дані.

Лістинг rate_limit.php:

```
<?php
ob_start();

$log_file = 'request.log';
$max_requests = 5;
$time_frame = 60;

$user_ip = $_SERVER['REMOTE_ADDR'];
$current_time = time();

$valid_requests = [];
$user_request_count = 0;

if (file_exists($log_file)) {
    $lines = file($log_file, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

    foreach ($lines as $line) {
        list($log_ip, $log_time) = explode(',', $line);

        if ($current_time - $log_time <= $time_frame) {
            $valid_requests[] = $line;

            if ($log_ip === $user_ip) {
                $user_request_count++;
            }
        }
    }
}

if ($user_request_count >= $max_requests) {
    http_response_code(429);

    echo "<h1>429 Too Many Requests</h1>";
    echo "<p style='color: red;'>Ви зробили занадто багато запитів (більше 5 за хвилину). Будь ласка, зачекайте та спробуйте пізніше.</p>";

    file_put_contents($log_file, implode(PHP_EOL, $valid_requests) . PHP_EOL);
} else {
    http_response_code(200);

    echo "<h1>200 OK</h1>";
    echo "<p style='color: green;'>Вітаємо! Це звичайний контент сторінки.</p>";
    echo "<p>Ви зробили <b>" . ($user_request_count + 1) . "</b> запит(ів) за останню хвилину з IP: $user_ip.</p>";

    $valid_requests[] = "$user_ip, $current_time";

    file_put_contents($log_file, implode(PHP_EOL, $valid_requests) . PHP_EOL);
}

ob_end_flush();
?>
```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		3

Результат:

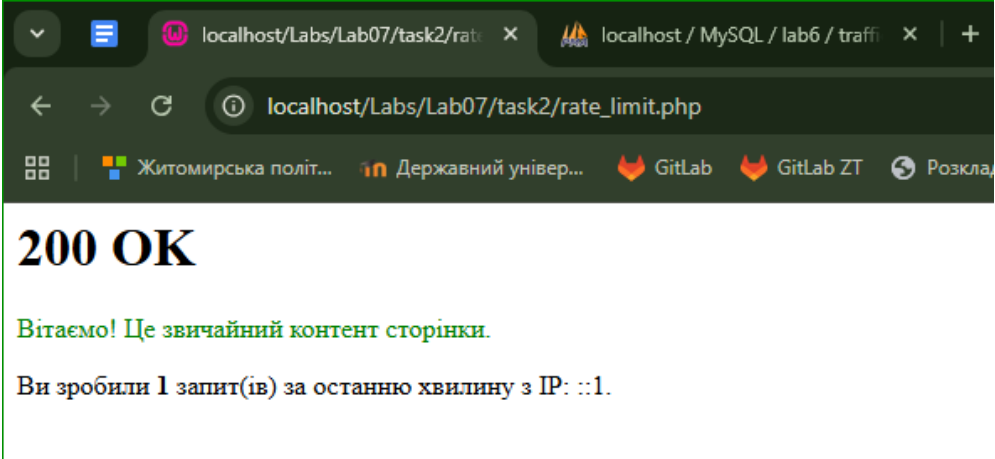


Рис. 2. Завдання 2

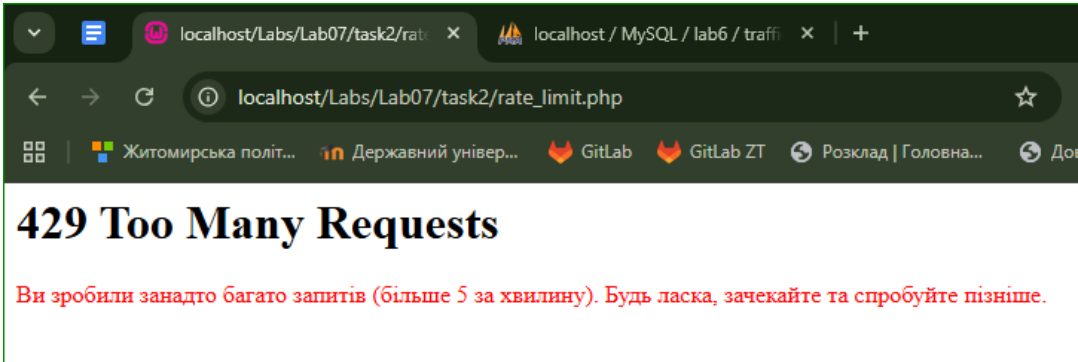


Рис. 3. Завдання 2

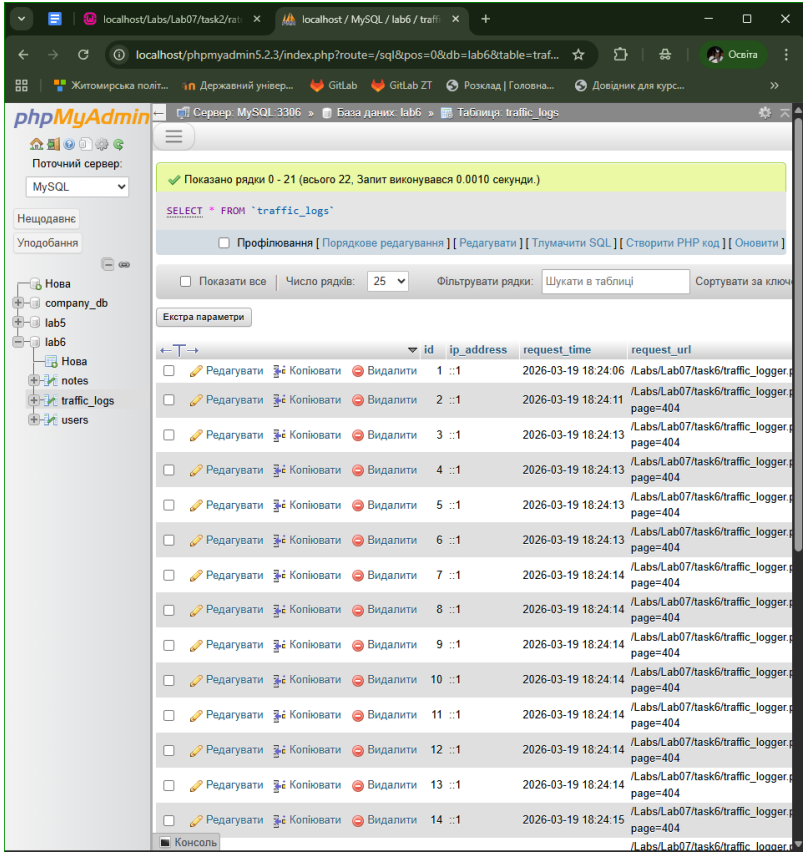


Рис. 4. Завдання 2

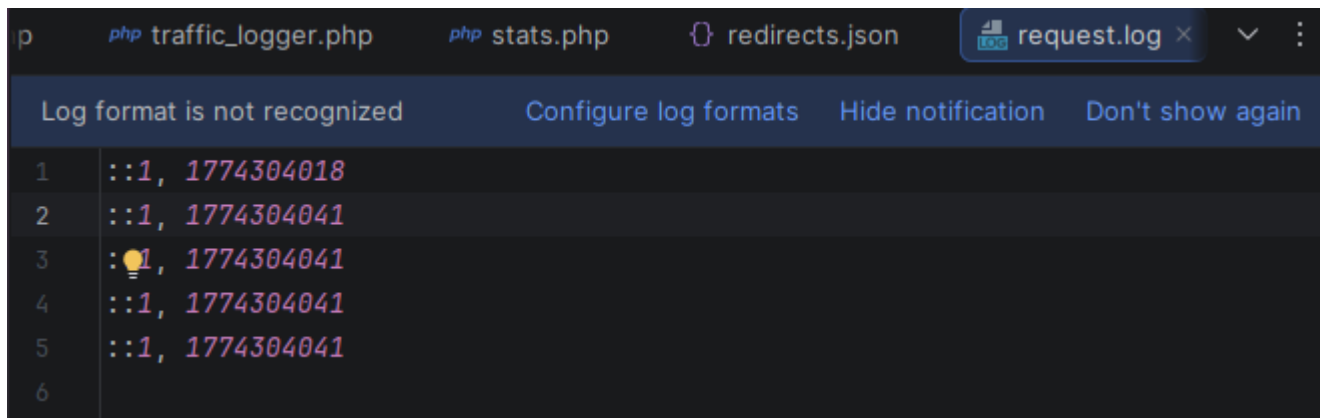


Рис. 5. Завдання 2

Завдання 3. перехоплення помилок та перенаправлення:

Створіть скрипт **error_handler.php**, який перехоплює фатальні помилки та замість стандартного повідомлення показує кастомну сторінку з відповідним статус-кодом.

Умови:

- Використовуйте **register_shutdown_function()**, щоб обробляти **E_ERROR**.
- Якщо сталася помилка:
 - Очистити буфер **ob_clean()**
 - Встановити статус **500 Internal Server Error**
 - Показати сторінку з вибаченням та часом, коли проблема буде вирішена.
- Якщо помилок немає, віддавати **200 OK**.

Підказка:

- **error_get_last()** допоможе отримати останню помилку.
- Використовуйте **ob_start()**, щоб уникнути передчасного виводу.

Лістинг **error_handler.php**:

```
<?php
ob_start();

function custom_fatal_error_handler()
{
    $error = error_get_last();

    if ($error !== null && $error['type'] === E_ERROR) {
        ob_clean();

        http_response_code(500);

        $estimated_fix_time = date('H:i', strtotime('+3 hour'));

        echo "<h1>500 - Внутрішня помилка сервера</h1>";
        echo "<p style='color: red;'>Вибачте, щось пішло не так. Ми вже працюємо над усуненням проблеми.</p>";
        echo "<p>Орієнтовний час відновлення роботи: <b>{$estimated_fix_time}</b>.</p>";

        ob_end_flush();
    } else {
        http_response_code(200);
    }
}
```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		5

```

        ob_end_flush();
    }
}

register_shutdown_function('custom_fatal_error_handler');

if (isset($_GET['crash']) && $_GET['crash'] == 1) {
    vavavavav();
}

echo "<h1>200 OK</h1>";
echo "<p style='color: green;'>Скрипт відпрацював ідеально, жодних фатальних поми-
лок не виявлено!</p>";
?>

```

Результат:

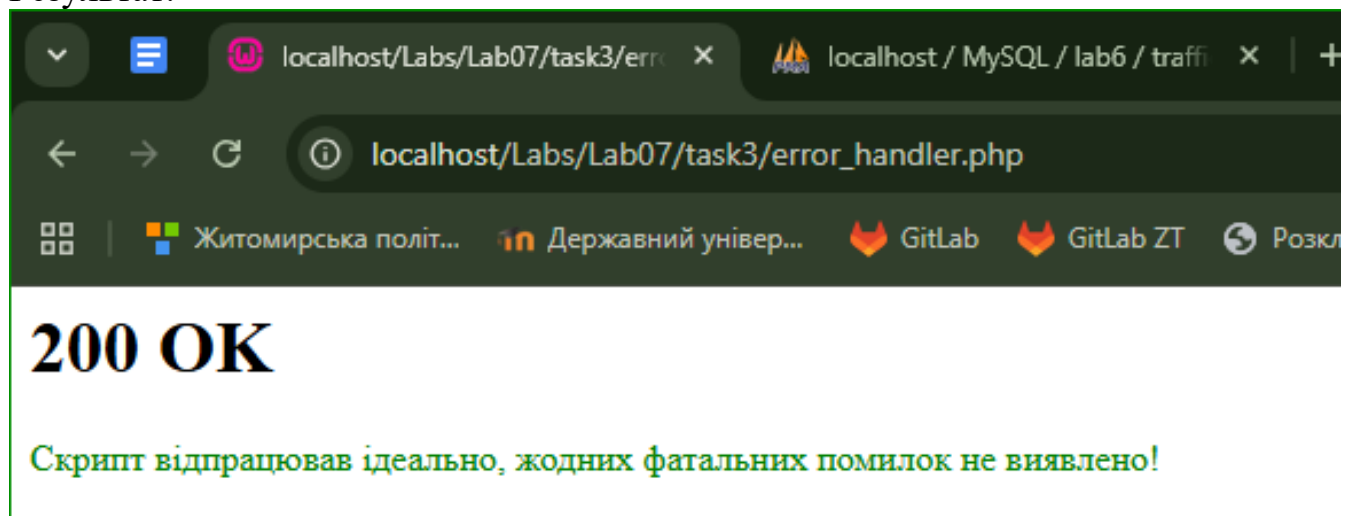


Рис. 6. Завдання 3

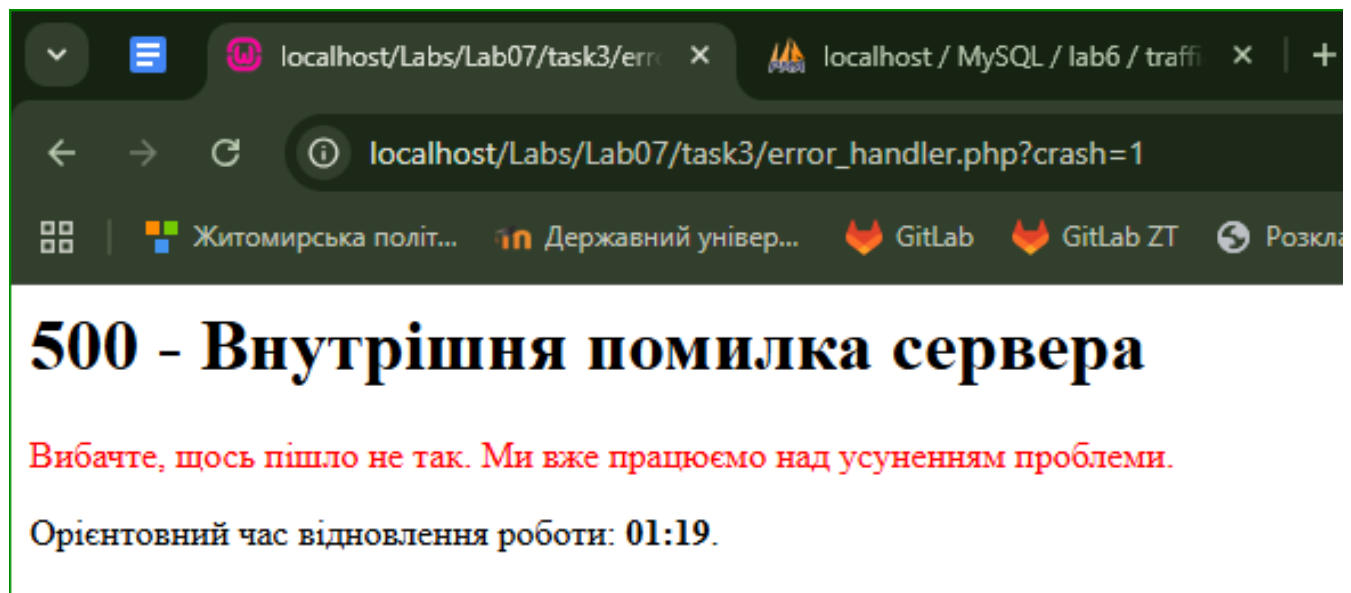


Рис. 7. Завдання 3

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 4. Гнучка система редиректів:

Створіть скрипт **redirect_manager.php**, який дозволяє керувати перенаправленнями через конфігураційний файл.

Умови:

- Є файл **redirects.json** зі списком перенаправлень:

```
{
    "/old-page": "/new-page",
    "/deprecated": "/404"
}
```

- Якщо користувач заходить на **/old-page**, його потрібно перенаправити на **/new-page** зі статусом **301 Moved Permanently**.
- Якщо сторінка **/deprecated**, видавати **404 Not Found** та відповідне повідомлення.
- Використовувати **ob_start()**, щоб контролювати вивід.

Підказка:

- json_decode(file_get_contents("redirects.json"), true);** дозволить отримати дані.
- header("Location: \$new_url", true, 301);** виконує перенаправлення.

Лістинг **redirect_manager.php**:

```
<?php
ob_start();

$json_file = 'redirects.json';

if (!file_exists($json_file)) {
    die("Помилка: Файл $json_file не знайдено!");
}

$redirects = json_decode(file_get_contents($json_file), true);

$current_page = isset($_GET['page']) ? $_GET['page'] : 1;

if (array_key_exists($current_page, $redirects)) {
    $target_url = $redirects[$current_page];

    if ($target_url === '/404') {
        http_response_code(404);
        echo "<h1>404 Not Found</h1>";
        echo "<p style='color: red;'>Ця сторінка застаріла і більше не підтримується (Deprecated).</p>";
    } else {
        $redirect_to = 'redirect_manager.php?page=' . $target_url;
        header("Location: " . $redirect_to, true, 301);
        exit();
    }
} else {
    http_response_code(200);

    if ($current_page === '/new-page') {
        echo "<h1>Вітаємо на новій сторінці!</h1>";
    }
}
```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        echo "<p style='color: green;*>Редирект 301 спрацював успішно. Ви були перенаправлені зі старої сторінки.</p>";
    } else {
        echo "<h1>Головна сторінка</h1>";
        echo "<p>Ви знаходитесь на сторінці: <b>$current_page</b></p>";
        echo "<p>Спробуйте перейти на: <a href='?page=/old-page'>/old-page</a> або <a href='?page=/deprecated'>/deprecated</a></p>";
    }
}

ob_end_flush();
?>

```

Лістинг redirects.json:

```

{
  "/old-page": "/new-page",
  "/deprecated": "/404"
}

```

Результат:

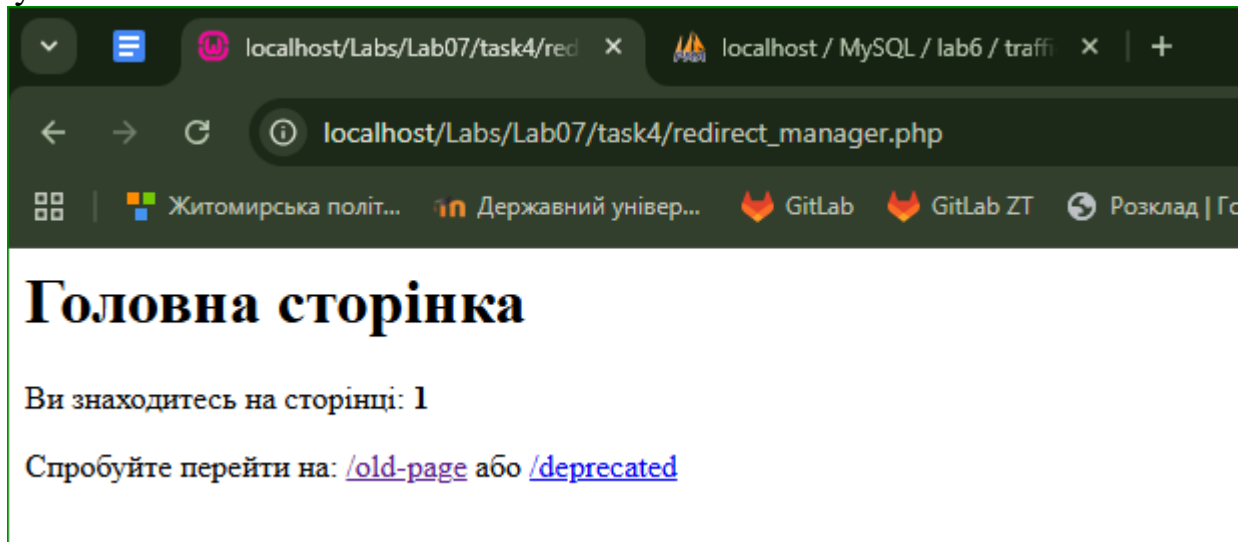


Рис. 8. Завдання 4

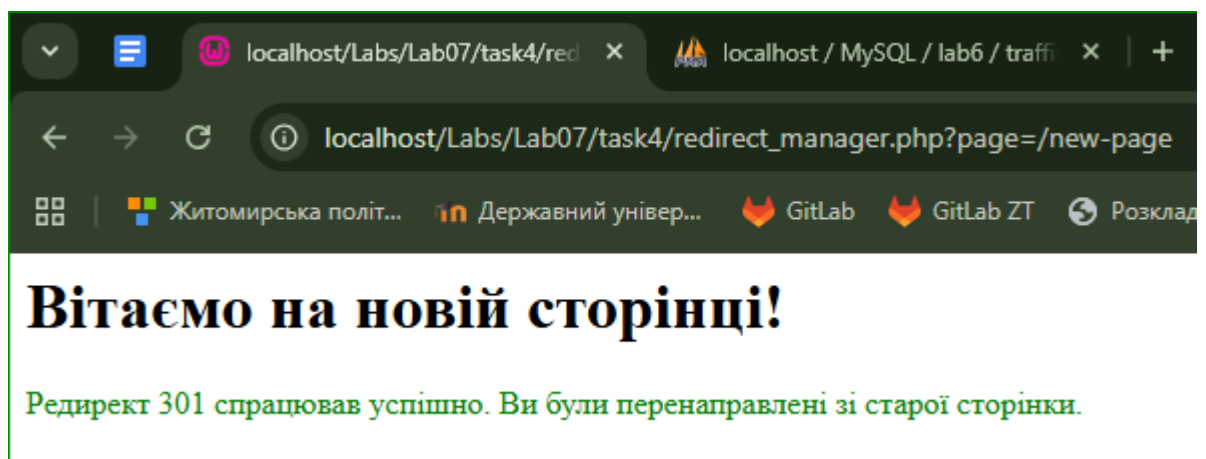


Рис. 9. Завдання 4

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		8

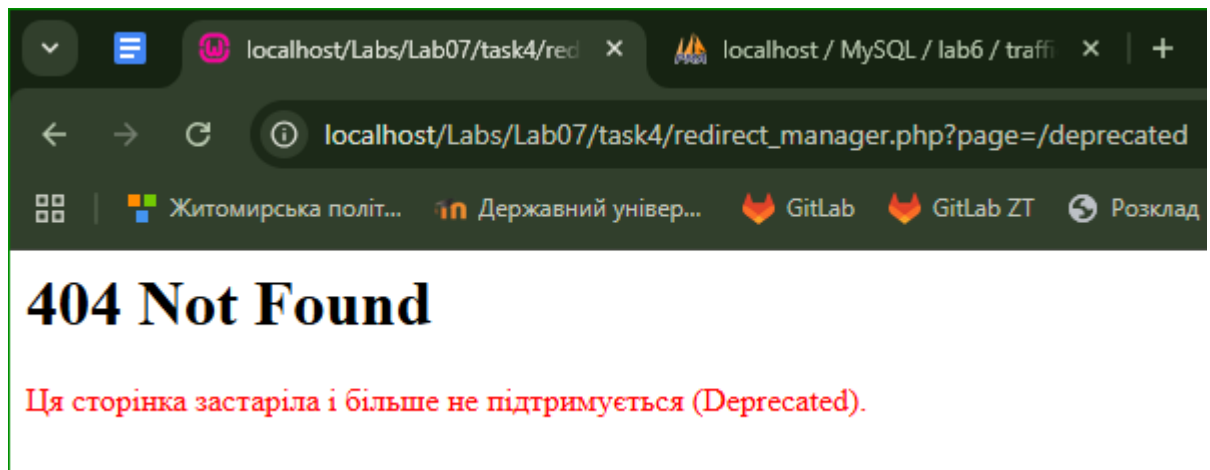


Рис. 10. Завдання 4

Завдання 5. Динамічне підключення заголовків без видимого виводу:

Створіть PHP-клас **Response**, який допомагає керувати HTTP-заголовками та буферизацією.

Функціонал класу:

- Метод **setStatus(\$code)**, який встановлює HTTP-статус.
- Метод **addHeader(\$header)**, який додає заголовок (наприклад, **Content-Type**).
- Метод **send(\$content)**, який очищає буфер, додає заголовки та відправляє відповідь.

Приклад використання:

```
$response = new Response();
$response->setStatus(200);
$response->addHeader("Content-Type: text/html");
$response->send("<h1>Вітаємо!</h1><p>Це динамічна відповідь.</p>");
```

Підказка:

- Використовуйте **header()** всередині методу **setStatus()**.
- Використовуйте **ob_start()**, **ob_clean()** перед виводом контенту.

Лістинг класу Response.php:

```
<?php
class Response {
    private $headers = [];

    public function __construct() {
        ob_start();
    }

    public function setStatus($code) {
        http_response_code($code);
    }

    public function addHeader($header) {
```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

```

        $this->headers[] = $header;
    }

    public function send($content) {
        ob_clean();

        foreach ($this->headers as $header) {
            header($header);
        }

        echo $content;

        ob_end_flush();
    }
}

echo "Цей текст ніхто не побачить: А О АО АООО ОАООАОА ООАОАО";

$response = new Response();
$response->setStatus(200);

$response->addHeader("Content-Type: text/html; charset=utf-8");
$response->addHeader("X-Custom-Header: My-PHP-APP");

$response->send("<h1>Vita!!!</h1><p>Це динамічна відповідь, згенерована через ООП
клас Response.</p>");
?>

```

Результат:

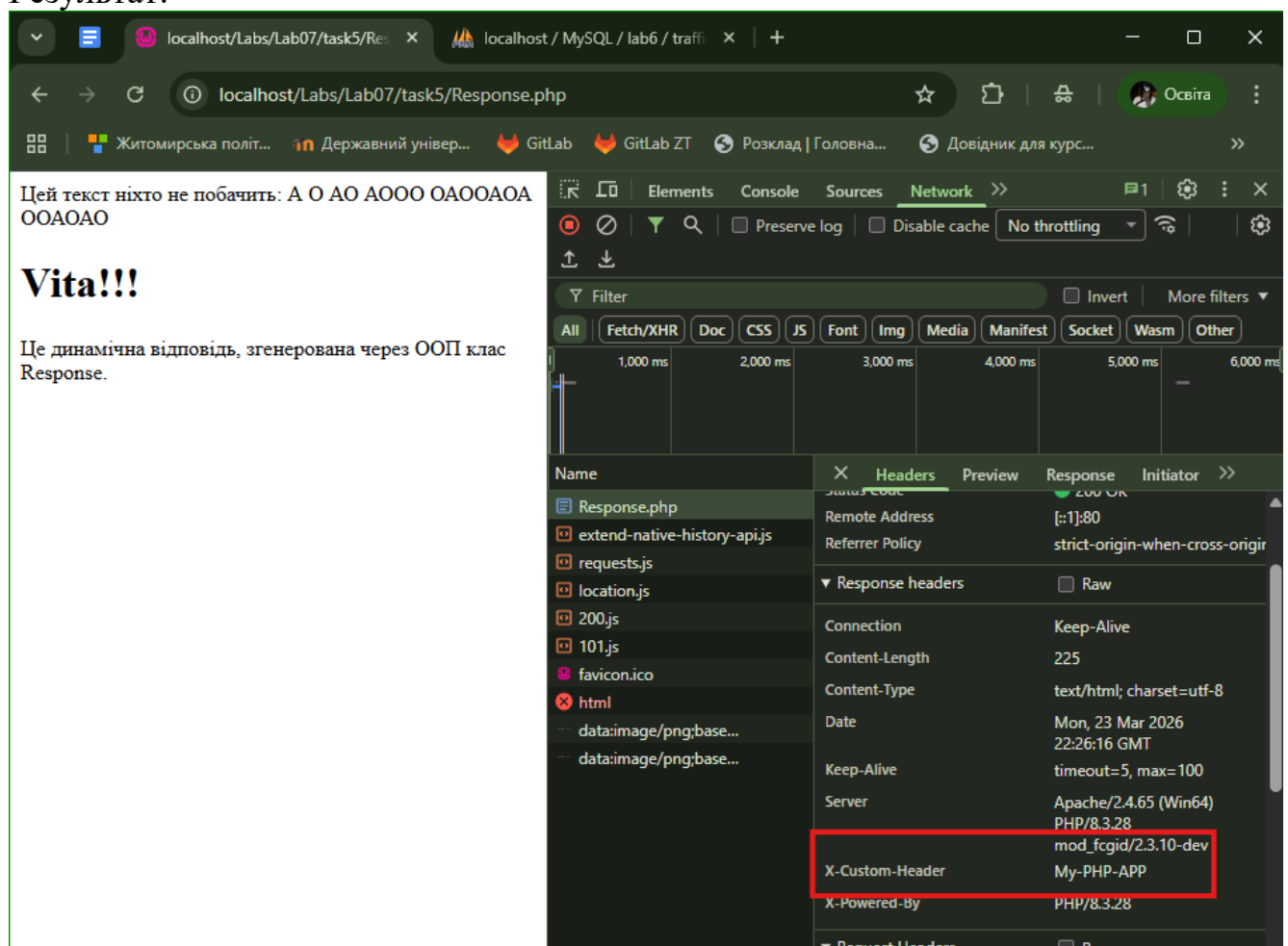


Рис. 11. Завдання 5

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк. 10
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 6. Аналіз трафіку та відповідей сервера:

Розробіть **traffic_logger.php**, який запише інформацію про всі запити в базу даних.

Що зберігати в БД?

- IP-адресу
- Час запиту
- Запитаний URL
- Відповідний HTTP-статус

Додаткові функції:

1. Реалізувати перегляд статистики через **stats.php**, який рахує кількість **404** помилок за останню добу.
2. Якщо відсоток **404** помилок перевищує 10% від загальної кількості запитів, надсилати адміністратору повідомлення.

Підказка:

- Використовуйте **MySQL (або SQLite)** для збереження даних.
- Функція **http_response_code()** допоможе отримати поточний статус.

Лістинг stats.php:

```
<?php
$host = "localhost";
$db = 'lab6';
$user = 'root';
$pass = '';
$pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8mb4", $user, $pass);

$yesterday = date("Y-m-d H:i:s", strtotime("-24 hours"));

$stmtTotal = $pdo->prepare("SELECT COUNT(*) FROM traffic_logs WHERE request_time
>= ?");
$stmtTotal->execute([$yesterday]);
$total_requests = $stmtTotal->fetchColumn();

$stmt404 = $pdo->prepare("SELECT COUNT(*) FROM traffic_logs WHERE request_time >=
? AND status_code = 404");
$stmt404->execute([$yesterday]);
$total_404 = $stmt404->fetchColumn();

$percentage_404 = 0;
if ($total_requests > 0) {
    $percentage_404 = ($total_404 / $total_requests) * 100;
}
?>

<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <title>Статистика сервера</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            padding: 20px;
        }
    </style>
</head>
<body>
    <div>
        <div>Загальна кількість запитів</div>
        <div>Кількість помилок 404</div>
        <div>Відсоток помилок 404</div>
    </div>
</body>
</html>
```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
    .stat-box {
        border: 1px solid #ccc;
        padding: 15px;
        border-radius: 5px;
        max-width: 400px;
        margin-bottom: 20px;
    }
    .alert {
        background-color: #f8d7da;
        color: #721c24;
        border: 1px solid #f5c6cb;
        padding: 15px;
        border-radius: 5px;
        font-weight: bold;
    }
    .table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    .table th, .table td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }
    .table th { background-color: #f2f2f2; }
</style>
</head>
<body>

<h1>Аналіз трафіку за останні 24 години</h1>

<div class="stat-box">
    <p>Загальна кількість запитів: <b><?= $total_requests ?></b></p>
    <p>Кількість помилок 404: <b><?= $total_404 ?></b></p>
    <p>Відсоток 404 помилок: <b><?= round($percentage_404, 2) ?>%</b></p>
</div>

<?php if ($percentage_404 > 10): ?>
    <div class="alert">
        ПОВІДОМЛЕННЯ ДЛЯ АДМІНІСТРАТОРА:<br>
        Увага! Кількість 404 помилок перевищує критичний поріг у 10% (зараз <?= round($percentage_404, 2) ?>%). <br>
        Можливо, на сайті є биті посилання або відбувається автоматизоване сканування (парсинг) вашого ресурсу.
    <?php
    ?>
    </div>
<?php else: ?>
    <p style="color: green;">Стан сервера в нормі. Відсоток помилок не перевищує ліміт.</p>
<?php endif; ?>

<h2>Останні запити:</h2>
<table class="table">
    <tr>
        <th>ID</th>
        <th>IP-адреса</th>
        <th>URL</th>
        <th>Статус</th>
        <th>Час</th>
    </tr>

```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

```

<?php
    $logs = $pdo->query("SELECT * FROM traffic_logs ORDER BY request_time DESC
LIMIT 10")->fetchAll();
    foreach ($logs as $log) {
        $color = ($log['status_code'] == 404) ? 'color: red;' :
(($log['status_code'] == 200) ? 'color: green;' : '');
        echo "<tr>
            <td>{$log['id']}</td>
            <td>{$log['ip_address']}</td>
            <td>{$log['request_url']}</td>
            <td style='{$color}' font-
weight:bold;>{$log['status_code']}</td>
            <td>{$log['request_time']}</td>
        </tr>";
    }
?>
</table>

</body>
</html>

```

Лістинг traffic_logger.php:

```

<?php
$host = "localhost";
$db = 'lab6';
$user = 'root';
$pass = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8mb4", $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    ]);
} catch (\PDOException $e) {
    die('Error connecting to SQL Server: ' . $e->getMessage());
}

function log_traffic() {
    global $pdo;

    $ip = $_SERVER['REMOTE_ADDR'];
    $url = $_SERVER['REQUEST_URI'];
    $time = date('Y-m-d H:i:s');

    $status = http_response_code();
    if (!$status) $status = 200;

    $stmt = $pdo->prepare("INSERT INTO traffic_logs (ip_address, request_time,
request_url, status_code) VALUES (?, ?, ?, ?)");
    $stmt->execute([$ip, $time, $url, $status]);
}

register_shutdown_function('log_traffic');

$page = $_GET['page'] ?? 'home';

if ($page === '404') {
    http_response_code(404);
    echo "<h1>404 Not Found</h1><p>Сторінку не знайдено.</p>";
} elseif ($page === '500') {
    http_response_code(500);
    echo "<h1>500 Server Error</h1><p>Помилка сервера.</p>";
} else {
    http_response_code(200);
}

```

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

```
echo "<h1>Головна сторінка</h1><p>Все працює чудово!</p>";  
}  
  
echo "<hr><p><em>Цей запит буде записано в базу даних після завершення заванта-  
ження сторінки.</em></p>";  
echo "<p>Спробуйте згенерувати 404 помилку: <a href='?page=404'>/404 сторі-  
нка</a></p>";  
?>
```

Результат:

Статистика сервера

localhost / MySQL / lab6 / traffi

localhost/Labs/Lab07/task6/stats.php

Освіта

Житомирська політ... Державний універ... GitLab GitLab ZT Розклад | Головна... Довідник для курс...

Аналіз трафіку за останні 24 години

Загальна кількість запитів: 0

Кількість помилок 404: 0

Відсоток 404 помилок: 0%

Стан сервера в нормі. Відсоток помилок не перевищує ліміт.

Останні запити:

ID	IP-адреса	URL	Статус	Час
22	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:25:45
21	::1	/Labs/Lab07/task6/traffic_logger.php	200	2026-03-19 18:25:41
20	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:16
14	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
15	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
16	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
17	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
18	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
19	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:15
7	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-19 18:24:14

Рис. 12. Завдання 6

localhost/Labs/Lab07/task6/trafi localhost / MySQL / lab6 / traffi

localhost/Labs/Lab07/task6/traffic_logger.php

Житомирська політ... Державний універ... GitLab GitLab ZT Розклад | Гол

Головна сторінка

Все працює чудово!

Цей запит буде записано в базу даних після завершення завантаження сторінки.

Спробуйте згенерувати 404 помилку: [/404 сторінка](#)

Рис. 13. Завдання 6

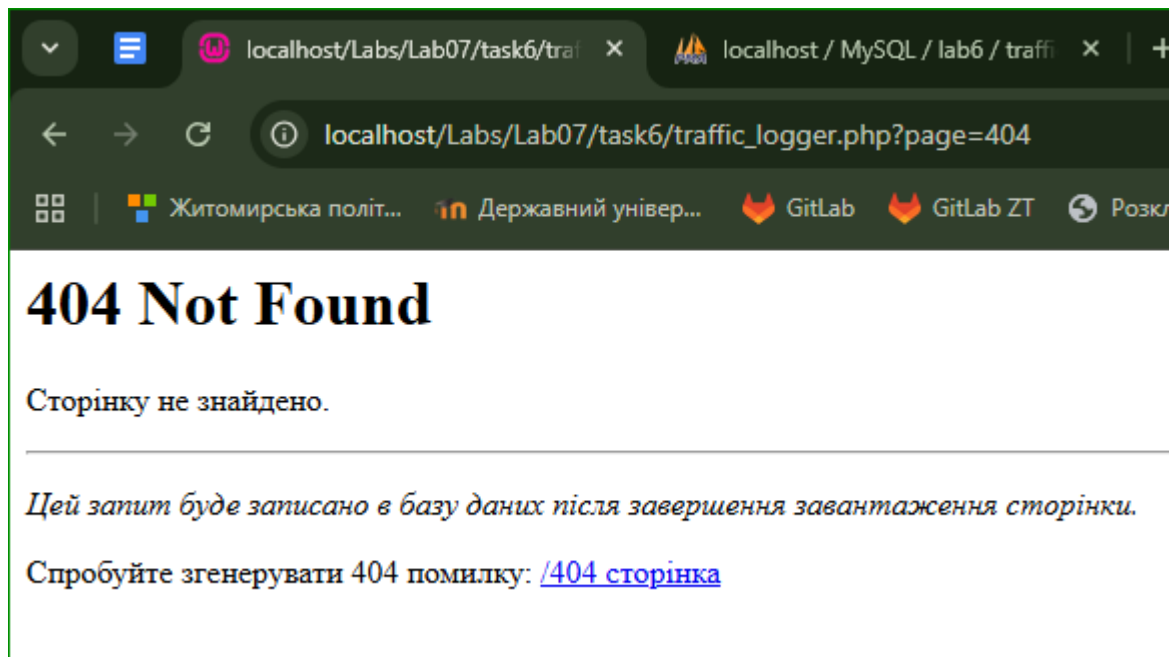


Рис. 14. Завдання 6

Статистика сервера

localhost / MySQL / lab6 / traffi

localhost/Labs/Lab07/task6/stats.php

Житомирська політ... Державний універ... GitLab GitLab ZT Розклад Головна... Довідник для курс...

Аналіз трафіку за останні 24 години

Загальна кількість запитів: 31

Кількість помилок 404: 30

Відсоток 404 помилок: 96.77%

ПОВІДОМЛЕННЯ ДЛЯ АДМІНІСТРАТОРА:
Увага! Кількість 404 помилок перевищує критичний поріг у 10% (зараз 96.77%).
Можливо, на сайті є биті посилання або відбувається автоматизоване сканування (парсинг) вашого ресурсу.

Останні запити:

ID	IP-адреса	URL	Статус	Час
53	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:30:00
52	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:58
51	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:58
50	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:58
48	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:57
49	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:57
47	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:57
45	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:56
43	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:56
42	::1	/Labs/Lab07/task6/traffic_logger.php?page=404	404	2026-03-23 22:29:56

Рис. 15. Завдання 6

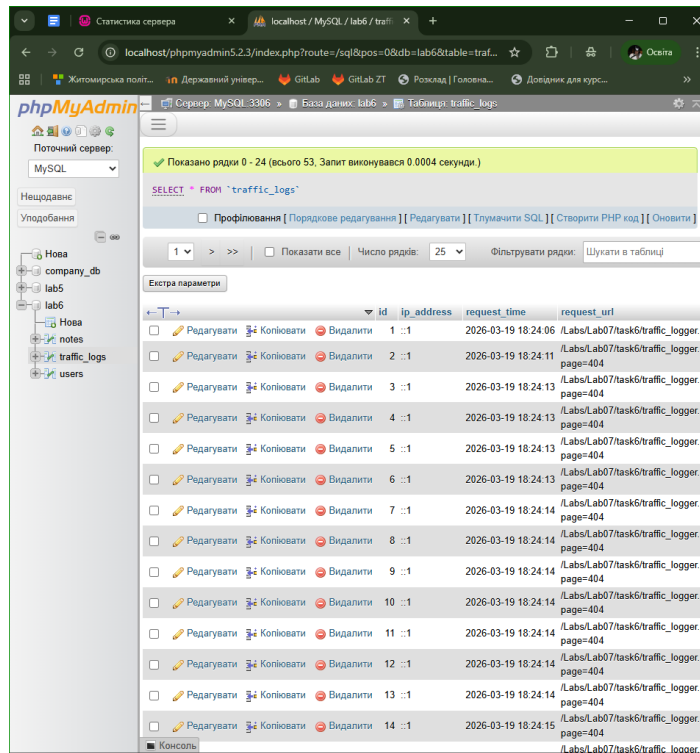


Рис. 16. Завдання 6. БД phpMyAdmin. Логування трафіку

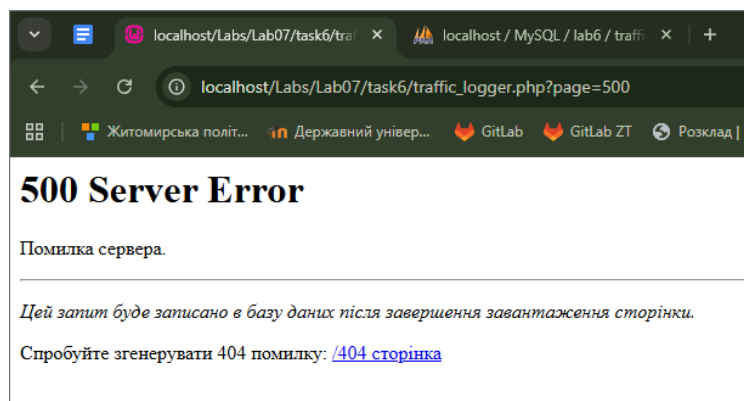


Рис. 17. Завдання 6. 500 Server Error

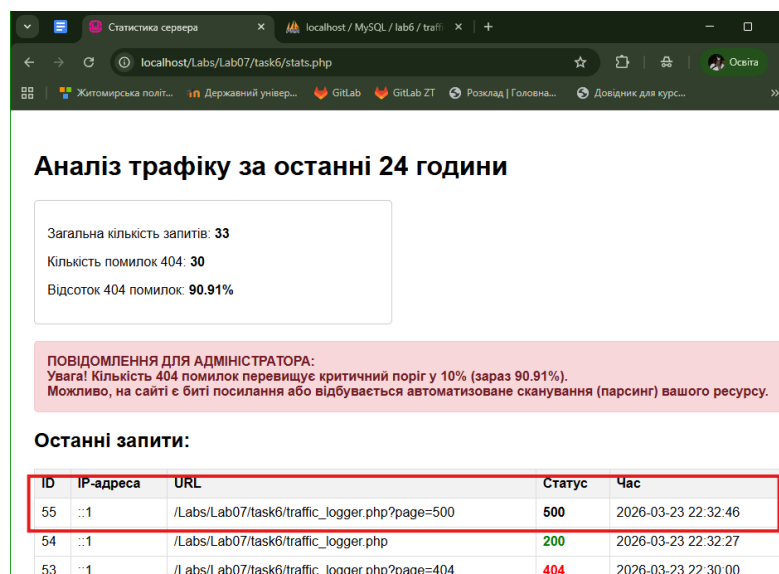


Рис. 18. Завдання 6

Висновок:

Під час виконання 7 лабораторної роботи я детально ознайомився з принципами взаємодії між клієнтом та сервером, глибоко вивчивши механізми керування HTTP-статусами та буферизацією виводу в PHP. На практиці я успішно реалізував систему динамічного кешування сторінок, навчившись зберігати згенерований контент за умови успішної відповіді сервера та коректно обробляти ситуації з відсутніми ресурсами. Важливим етапом стало створення захисту від перевантажень (Rate Limiting), де я застосував алгоритми обмеження кількості запитів для одного IP з автоматичною видачею статусу 429. Крім того, я розробив надійний обробник фатальних помилок, який за допомогою функцій керування буфером дозволяє елегантно перехоплювати збої та виводити користувачу кастомну сторінку з кодом 500. Також я налаштував гнучку систему маршрутизації, закріпивши навички роботи з 301-ми редиректами через конфігураційні файли у форматі JSON. Застосовуючи об'єктно-орієнтований підхід, я спроектував власний клас Response для зручного інкапсульованого керування HTTP-заголовками та відправкою даних. На завершення роботи я реалізував повноцінний аналізатор серверного трафіку, який автоматично фіксує всі запити у базі даних MySQL та розраховує статистику помилок для своєчасного сповіщення адміністратора. У результаті я здобув цінний досвід серверної оптимізації, який дозволить мені створювати більш безпечні, швидкі та стійкі до навантажень вебдодатки.

		Пригорницький О. В.			ДУ «Житомирська політехніка».26.121.04.07 – Лр7	Арк.
		Ковтун В. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		17